A GRASP based approach for the Bin Packing Problem with General Precedence Constraints

Carlos E. Aguiar[1], **Pedro Saccomani**[1], **Rangel Á. Barroso**[1]

[1]Departamento de Ciência da Computação
Universidade Federal de Minas Gerais (UFMG) – Belo Horizonte, MG – Brazil

Caduaguiar.roma@gmail.com, psaccomani15@gmail.com, rangel-avila@ufmg.br

1. Introdução

Em ambientes modernos de desenvolvimento de software, organizações enfrentam desafios cada vez mais complexos ao gerenciar a entrega de funcionalidades ao longo de múltiplos ciclos de desenvolvimento e janelas de deploy.

Equipes de software precisam decidir regularmente como agrupar e programar funcionalidades para entrega, respeitando dependências complexas entre componentes, módulos e funcionalidades. Esse cenário real reflete um clássico desafio de otimização combinatória, onde recursos limitados (janelas de entrega, capacidade de desenvolvimento, ambientes de teste) devem ser alocados de forma eficiente enquanto se mantém a ordem correta de execução de tarefas interdependentes.

A complexidade se intensifica quando se considera que os recursos frequentemente possuem tamanhos, prioridades e relações de precedência que não podem ser violadas durante o processo de entrega. Considerando esses fatores, apresenta-se a definição formal do problema.

O Problema de Bin Packing com Restrições Gerais de Precedência (BPP-GP) pode ser formalmente definido da seguinte forma: considere um conjunto de itens $N = \{1, 2, \ldots, n\}$, onde cada item $j \in N$ possui um peso $w_j \in \mathbb{R}^+$. Seja $M = \{1, 2, \ldots, m\}$ o conjunto de caixas (bins) disponíveis, cada uma com capacidade $C \in \mathbb{R}^+$. O problema consiste em atribuir cada item j a exatamente uma caixa $i_j \in M$, sem violar a capacidade de nenhuma caixa.

$$\sum_{j=1} w_j x_{jk} \leq C, \quad \forall k \in M,$$

$$x_{jk} = \begin{cases} 1, & \text{se o item } j \text{ é alocado à caixa } k, \\ 0, & \text{caso contrário.} \end{cases}$$

Além das restrições de capacidade, assume-se a existência de um conjunto de precedências gerais P, formado por tuplas (a, b, t_{ab}), onde $a, b \in N$ e $t_{ab} \in \mathbb{Z}$. Cada tupla define uma restrição do tipo:

$$i_a - i_b \geq t_{ab}.$$

Essa relação estabelece que o item a deve ser alocado a uma caixa que esteja pelo menos t_{ab} posições à frente da caixa de b. Valores de t_{ab} determinam diferentes níveis de separação:

- $t_{ab} = 0$: o item a não pode ser alocado em uma caixa anterior a b;
- $t_{ab} = 1$: o item a deve estar em uma caixa estritamente posterior à de b;
- $t_{ab} > 1$: exige-se uma separação mínima de t_{ab} caixas entre b e a.

Assim, as restrições de precedência impõem uma ordem parcial geral entre os itens, válida tanto dentro quanto entre caixas.

O objetivo do BPP-GP é determinar o menor número de caixas m necessárias para alocar todos os itens, respeitando simultaneamente as restrições de capacidade e todas as relações de precedência:

$$\min m.$$

O BPP-GP pertence à classe de problemas de otimização NP-Difícil. Essa classificação decorre do fato de que ele é uma extensão do problema de Bin Packing clássico (BPP), que já é NP-Difícil. A adição de restrições adicionais permite descrever problemas ainda mais complexos. A versão de decisão desse problema, que pergunta se existe um empacotamento viável usando no máximo k caixas enquanto respeita todas as restrições, é NP-Completa, tornando improvável a existência de algoritmos exatos polinomiais para resolver grandes instâncias.

Vários outros problemas relacionados compartilham similaridades estruturais com o BPP-GP, incluindo o Job Shop Problem com restrições de precedência, tratado por[Flórez et al. 2013] e o Problema de Programação de Projetos com Recursos Restritos (RCPSP) [Goncharov 2025].

Além disso, variantes como o Bin Packing com Conflitos (onde certos itens não podem ser colocados na mesma caixa) [Ekici 2023] e o Bin Packing Multidimensional [Khan et al. 2021] oferecem insights sobre algumas maneiras de lidar com as restrições.

O BPP-GP apresenta diversas características únicas que o distinguem de problemas clássicos de otimização combinatória. Restrições de precedência criam uma ordenação parcial que pode resultar em grafos de dependência complexos, possivelmente incluindo ramificações paralelas e múltiplos níveis hierárquicos. Em contextos de entrega de software, essas restrições podem representar dependências técnicas, prioridades de negócio, relações entre user stories ou requisitos de testes. O problema permite precedência intra-caixa (dependências dentro da mesma janela de entrega) e inter-caixa (dependências que atravessam janelas), complicando significativamente o espaço de solução e exigindo mecanismos sofisticados de propagação de restrições.

Esse problema merece investigação devido à sua aplicabilidade direta em desafios críticos da engenharia de software moderna e gestão de projetos. À medida que sistemas de software se tornam cada vez mais complexos e equipes adotam metodologias ágeis com ciclos frequentes de entrega, a alocação eficiente de recursos em janelas de entrega respeitando dependências torna-se fundamental para o sucesso do projeto. Um escalonamento inadequado pode causar gargalos no desenvolvimento, atrasos na liberação, uso ineficiente de recursos e acúmulo de dívida técnica.

Além disso, investigar uma abordagem meta-heurística para esse problema pode avançar o campo mais amplo da otimização combinatória com restrições, oferecendo insights aplicáveis a outros domínios, como cadeia de suprimentos, escalonamento industrial e problemas de alocação de recursos.

Esta pesquisa se baseia na metodologia proposta por [Kramer et al. 2017] para o BPP-GP. A abordagem original fornece um arcabouço estruturado para lidar com empacotamento com precedência por meio de técnicas iterativas de busca local. Neste trabalho, utilizamos este baseline como ponto de partida e o estendemos implementando uma meta-heurística baseada em GRASP, visando aumentar a eficiência da propagação das precedências e das estratégias de empacotamento. O objetivo permanece minimizar o número de caixas, garantindo viabilidade com respeito às dependências intra e inter-caixas, aproveitando as vantagens do baseline enquanto exploramos melhorias por meio do GRASP.

A justificativa para este trabalho reside tanto em sua relevância prática quanto na contribuição teórica que pode oferecer. Na prática, a otimização da entrega de recursos de software sob restrições de precedência atende a uma necessidade real enfrentada por equipes de desenvolvimento, onde a complexidade do escalonamento manual frequentemente leva a ineficiências e atrasos. Do ponto de vista teórico, a formulação e análise de heurísticas para o BPP-GP expandem o corpo de conhecimento em otimização combinatória sob restrições complexas, fornecendo novas perspectivas que podem ser aplicadas além da engenharia de software, como logística, manufatura e gestão de projetos. Assim, a pesquisa se justifica por gerar impacto direto em processos críticos de engenharia, enquanto fortalece o entendimento acadêmico de problemas NP-Difíceis com restrições adicionais.

2. Trabalhos Relacionados

O Bin Packing com Restrições de Precedência (BPP-P) pode ser visto como uma extensão do conhecido Bin Packing Clássico para incorporar restrições de precedência. Uma instância do BPP-P é descrita por um conjunto de itens $N = 1, 2, \ldots, n$, com pesos não negativos, e um conjunto de caixas $M = 1, 2, \ldots, m$, cada uma com capacidade C. Além disso, no conjunto de itens existe uma relação de **precedência**. O objetivo é empacotar todos os itens nas caixas respeitando a capacidade e a precedência: itens com menor precedência devem ser alocados em caixas de menor índice.

Uma solução para esse problema foi introduzida no trabalho de [Dell'Amico et al. 2012]. Eles consideram diferentes abordagens independentes: um procedimento de branch-and-bound, uma heurística construtiva gulosa (first fit) combinada com busca local, e também uma *Variable Neighborhood Search*.

Focando no problema clássico, no mesmo ano o trabalho de [Layeb and Chenche 2012] aborda o Bin Packing Unidimensional (1-BPP), a forma mais simples do problema. Os autores propõem uma nova abordagem baseada na metaheurística GRASP (Greedy Randomized Adaptive Search Procedure). A metodologia é implementada em duas fases: a primeira é uma fase construtiva que utiliza uma nova heurística gulosa randomizada, sendo uma versão híbrida das heurísticas First Fit (FF) e Best Fit (BF); a segunda fase consiste em uma busca local baseada no algoritmo Tabu Search (TS), usada para refinar e melhorar a solução encontrada na primeira fase. Os resultados experimentais, conduzidos em instâncias benchmark (fáceis, médias e difíceis), demonstram que o GRASP proposto supera significativamente a heurística gulosa simples, alcançando resultados muito próximos das melhores soluções conhecidas.

O trabalho de [Pereira 2016] propõe uma heurística baseada em programação dinâmica e um método exato branch-and-bound, incorporando novos limites inferiores e regras de dominância para melhorar o desempenho. Os resultados computacionais mostram grande efetividade dos métodos, que encontraram soluções ótimas para todas as instâncias previamente abertas do benchmark clássico, superando abordagens anteriores tanto em qualidade quanto em velocidade. O estudo identifica que instâncias com pesos grandes e poucas relações de precedência (baixa força de ordem) são as mais desafiadoras.

Mais tarde, [Kramer et al. 2017] propõem uma solução para o BPP-GP, que também é o foco deste trabalho. Esse problema generaliza ainda mais o BPP-P ao introduzir precedências com pesos: cada relação entre itens a e b é descrita por um tupla (a, b) com um inteiro associado t, e o objetivo é ter $i_a - i_b \geq t$, onde i_a e i_b são as caixas onde a e b são inseridos. Esse trabalho introduz também um algoritmo simples baseado em *Iterated Local Search*. Usando heurísticas construtivas independentes para construir a solução inicial e então refiná-la. Os resultados computacionais mostram que o método é robusto e gera soluções de alta qualidade dentro de tempo razoável, mesmo quando comparado com métodos de ponta dedicados a casos particulares do problema. Os autores concluem que a abordagem é efetiva, embora a introdução de precedências heterogêneas apresente novos desafios e deixe muitas instâncias sem solução ótima provada, sugerindo direções para pesquisas futuras.

Expandindo a aplicação de heurísticas em problemas semelhantes,

[Lalaoui and El Afia 2018] abordam o Problema de Balanceamento de Linha de Montagem do tipo I (SALBP-I) propondo uma meta-heurística adaptativa. A abordagem usa um algoritmo de resfriamento simulado generalizado, cujos parâmetros são ajustados dinamicamente por um sistema fuzzy, buscando encontrar rapidamente soluções eficazes para rebalancear linhas de produção em resposta a fatores como flutuação de demanda. Os autores demonstram, em benchmark conhecido, que o método é eficaz para minimizar o número de estações dado um tempo de ciclo.

A literatura existente apresenta métodos robustos para o BPP-GP e variantes. Notavelmente, [Kramer et al. 2017] propõem um método baseline eficiente baseado em Iterated Local Search (ILS). Paralelamente, outras meta-heurísticas têm demonstrado grande sucesso em problemas semelhantes, como GRASP, aplicado com sucesso por [Layeb and Chenche 2012] ao 1-BPP clássico. Enquanto a abordagem ILS de [Kramer et al. 2017] serve como referência consolidada, o sucesso de GRASP em otimização combinatória motiva sua aplicação ao BPP-GP.

Este projeto busca estender o baseline de [Kramer et al. 2017] desenvolvendo e integrando uma meta-heurística GRASP adaptada, substituindo o método ILS original para explorar um novo caminho de solução para esse problema complexo.

3. Metodologia

Neste trabalho, foi implementada a abordagem GRASP que utiliza ideias do trabalho de [Kramer et al. 2017]. Foi utilizado esse estudo como método de controle. Os autores propõem um algoritmo baseado em Iterated Local Search, apresentado abaixo:

Algorithm 1: Meta-heurística de controle

```
function BM-ILS(Instance):
    ImprovedInstance ← PreProcessing(Instance)
    LB ← ComputeBounds(ImprovedInstance)
    /* Construção da primeira solução                    */
    x_incumbent ← ConstructInitialSolution(ImprovedInstance)
    x_best ← x_incumbent
    repeat
        /* Intensificação usando batching moves          */
        x_incumbent ← BM − LS(x_incumbent) if z(x_incumbent) < z(x_best) then
            x_best ← x_incumbent
        ρ ← random(1, K_MAX(ImprovedInstance))
        x_incumbent ← DestroyShrinkRepair(x_incumbent, ρ)   // Diversificação
    until z(x_best) = LB or Timeout()
    return x_best
```

Esse algoritmo possui três fases principais:

1. Pré-processamento e cálculo do limite inferior: essa fase acelera a convergência do algoritmo. Eles adaptam procedimentos apresentados em [Dell'Amico et al. 2012].
2. Cálculo da solução inicial: os autores utilizam duas heurísticas construtivas gulosas independentes e uma abordagem de programação inteira linear. Ao final, escolhe-se a melhor solução.
3. Busca local: essa fase refina a solução inicial até atingirmos um tempo limite ou o limite inferior.

A abordagem GRASP construída é derivada desse estudo prévio. Cada iteração da meta-heurística GRASP contém:

- Construção da solução inicial: uma solução viável é construída usando um algoritmo guloso adaptativo randomizado.
- Passo de busca local: essa solução é refinada até alcançar um ótimo local.

Esse algoritmo guloso randomizado é controlado por α, que define a proporção entre exploração e intensificação. Em nosso trabalho, α será adaptativo.

Na etapa construtiva, duas heurísticas gulosas usadas por [Kramer et al. 2017] foram adaptadas para incorporar randomização. Além disso, avaliações intermediárias foram realizadas para decidir como tratar o uso das heurísticas construtivas, tentando responder a seguinte pergunta: "Deve-se tomar como referência [Kramer et al. 2017] e executar ambas as abordagens, selecionando o melhor resultado obtido, ou considerar igualmente a introdução de aleatoriedade nesse processo de escolha?". Outras heurísticas potenciais também foram exploradas, e na busca local, algoritmo BM-ILS apresentado anteriormente foi adotado.

Espera-se que o trabalho produza soluções pelo menos comparáveis às de [Kramer et al. 2017] dado o mesmo tempo de execução, e possivelmente melhores em alguns benchmarks devido ao fator exploratório/adaptativo do GRASP.

3.1. Avaliação Experimental

A avaliação experimental será estruturada para permitir uma comparação rigorosa, reprodutível e estatisticamente fundamentada entre a abordagem GRASP proposta e métodos consolidados para o BPP-GP. O protocolo descrito nesta subseção segue práticas comuns em estudos de heurísticas e meta-heurísticas.

A avaliação seguirá o seguinte protocolo:

- **Métricas de desempenho:** número de caixas usadas, gap médio para limites inferiores conhecidos, distribuição das soluções em múltiplas execuções. Eficiência computacional será medida via tempo de CPU, número de iterações e comportamento de convergência.
- **Análise estatística:** devido ao caráter estocástico do GRASP, cada experimento será repetido diversas vezes (ex.: 10 runs independentes). Serão reportados média, desvio padrão e intervalos de confiança.
- **Instâncias benchmark:** usaremos os conjuntos estendidos de [Kramer et al. 2017] e o conjunto original de [Otto et al. 2013], cobrindo ampla gama de tamanhos e complexidades.
- **Ajuste de parâmetros:** análise de sensibilidade para α, buscando identificar configurações ótimas.
- **Estudos de ablação:** removendo ou modificando componentes do GRASP (heurísticas randomizadas, BM-ILS, α adaptativo) para medir impacto individual.
- **Ambiente experimental:** descrição do hardware, sistema operacional, linguagem utilizada e configurações de compilação para garantir reprodutibilidade.
- **Critérios de parada:** tempo limite comum a todos os métodos, número máximo de iterações e parada antecipada ao atingir o limite inferior.
- **Replicação e controle de aleatoriedade:** múltiplas execuções por instância, sementes documentadas e registro dos melhores, piores e medianos resultados.
- **Reprodutibilidade:** disponibilização de código, logs e scripts utilizados na condução dos experimentos.

Esse framework de avaliação estabelece bases sólidas para uma comparação justa e cientificamente robusta entre o GRASP proposto e os métodos de referência. Ele garante consistência metodológica, mensuração confiável do desempenho e análise crítica das escolhas de projeto feitas ao longo da construção da meta-heurística.

4. Resultados

Nesta seção são apresentados os resultados experimentais obtidos pela abordagem GRASP proposta. A análise compara o desempenho do GRASP com o baseline de [Kramer et al. 2017], considerando qualidade das soluções e eficiência computacional.

4.1. Configuração Experimental

A Tabela 1 resume o ambiente utilizado para todos os testes.

Table 1. Configuração experimental utilizada nos testes.

Hardware	**Especificar CPU, RAM, frequência**
Sistema Operacional	**Inserir versão**
Linguagem	**C++/Python + versão do compilador/interpreter**
Tempo limite por execução	**Inserir limite**
Número de execuções por instância	**Inserir número (ex.: 10)**
Semente base	**Inserir semente**

As instâncias utilizadas incluem o conjunto estendido de [Kramer et al. 2017] e o conjunto original de [Otto et al. 2013]. No total, foram selecionadas **7350 instâncias utilizadas**.

4.2. Qualidade das Soluções

4.2.1. Comparação do número de bins

A Tabela 2 apresenta a média do número de caixas utilizadas pelo GRASP e pelo baseline, acompanhada do desvio padrão e dos valores mínimo/máximo. **acrescentar comentários sobre a tabela abaixo.**

Table 2. Comparação do número de caixas utilizadas.

Instância	GRASP (média)	Baseline	Gap Médio	Melhor Método
Instância 1	**x**	**y**	**z**	**GRASP/Baseline**
Instância 2	...			



Figure 1. Comparação entre número médio de caixas utilizadas.

4.2.2. Gap para limites inferiores

A Tabela 3 sumariza os valores médios do gap em relação ao limite inferior calculado na fase de pré-processamento. **acrescentar comentários sobre a tabela abaixo.**

Table 3. Gap médio para os limites inferiores.

Instância	LB	Gap GRASP	Gap Baseline
Instância 1	**a**	**b**	**c**
...			

4.3. Eficiência Computacional

A Tabela 4 apresenta o tempo médio de CPU. **acrescentar comentários sobre a tabela abaixo.**

Table 4. Tempo médio de execução (em segundos).

Instância	GRASP	Baseline	Diferença
Instância 1	**x**	**y**	**z**
...			

acrescentar comentários sobre o gráfico abaixo.



Figure 2. Comportamento de convergência do GRASP.

4.4. Análise de Sensibilidade de α

A Figura 3 e a Tabela 5 sumarizam o impacto de valores diferentes de α sobre a qualidade das soluções. **acrescentar comentários sobre a tabela e imagem abaixo.**

Table 5. Impacto de α no desempenho.

α	Média de caixas	Tempo médio	Gap
0.0	**x**	**y**	**z**
0.5	...		
Adaptativo	...		



Figure 3. Comportamento do GRASP para diferentes valores de α.

4.5. Estudos de Ablação

Para avaliar a contribuição individual de cada componente, foi conduzido um estudo de ablação no qual variou-se sistematicamente a estrutura do método. Foram consideradas as seguintes variantes: (i) GRASP completo, utilizado como baseline; (ii) versão sem randomização nas heurísticas construtivas; (iii) versão sem o mecanismo de α adaptativo; (iv) versão sem o BM-ILS, empregando apenas busca local simples; e (v) configuração composta apenas pelo BM-ILS inicializado de forma determinística. Essa análise permite quantificar o impacto de cada componente na qualidade final das soluções.

Os resultados são apresentados na Tabela 6. **acrescentar comentários sobre a tabela abaixo.**

Table 6. Estudo de ablação: impacto dos componentes.

Configuração	Caixas (média)	Gap	Tempo
GRASP completo	**x**	**y**	**z**
Sem randomização	...		
Sem α adaptativo	...		

4.6. Discussão dos Resultados

Nesta seção final são discutidos os padrões observados:

- O GRASP apresentou desempenho **melhor/igual/pior** na maior parte das instâncias.

- A exploração controlada pela adaptação de α impactou positivamente/negativamente **(preencher)**.
- O BM-ILS contribuiu principalmente nas instâncias de alta densidade/alta precedência **(preencher)**.

5. Conclusões

Conclusões aqui.

References

Dell'Amico, M., Díaz, J. C. D., and Iori, M. (2012). The bin packing problem with precedence constraints. *Operations Research*, 60(6):1491–1504.

Ekici, A. (2023). A large neighborhood search algorithm and lower bounds for the variable-sized bin packing problem with conflicts. *European Journal of Operational Research*, 308(3):1007–1020.

Flórez, E., Gómez, W., and Bautista, L. (2013). An ant colony optimization algorithm for job shop scheduling problem. *arXiv preprint arXiv:1309.5110*.

Goncharov, E. (2025). A hybrid heuristic algorithm for the resource-constrained project scheduling problem. *arXiv preprint arXiv:2502.18330*.

Khan, A., Sharma, E., and Sreenivas, K. (2021). Geometry meets vectors: Approximation algorithms for multidimensional packing. *arXiv preprint arXiv:2106.13951*.

Kramer, R., Dell'Amico, M., and Iori, M. (2017). A batching-move iterated local search algorithm for the bin packing problem with generalized precedence constraints. *International Journal of Production Research*, 55(21):6288–6304.

Lalaoui, M. and El Afia, A. (2018). A fuzzy generalized simulated annealing for a simple assembly line balancing problem. *IFAC-PapersOnLine*, 51(32):600–605.

Layeb, A. and Chenche, S. (2012). A novel grasp algorithm for solving the bin packing problem. *International Journal of Information Engineering and Electronic Business*, 4(2):8.

Otto, A., Otto, C., and Scholl, A. (2013). Systematic data generation and test design for solution algorithms on the example of salbpgen for assembly line balancing. *European Journal of Operational Research*, 228(1):33–45.

Pereira, J. (2016). Procedures for the bin packing problem with precedence constraints. *European Journal of Operational Research*, 250(3):794–806.